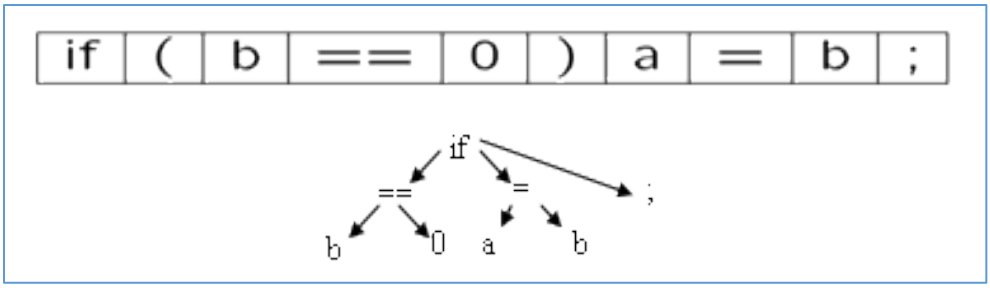
Chapter 3

Syntax Analysis

Syntax Analysis

Check syntax and construct abstract syntax tree



- Error reporting and recovery
- Model using context free grammars
- Recognize using Push down automata/Table Driven Parsers

This is the second phase of the compiler. In this phase, we check the **syntax** and construct the abstract syntax tree. This phase is modeled through context free grammars and the structure is recognized through **push down automata** or **table-driven parsers.** The syntax analysis phase verifies that the string can be generated by the grammar for the source language. In case of any syntax errors in the program, the parser tries to report as many errors as possible. Error reporting and recovery form a very important part of the syntax analyzer. The error handler in the parser has the following goals: .

- It should report the presence of errors clearly and accurately.
- It should recover from each error quickly enough to be able to detect subsequent errors. .
- It should not significantly slow down the processing of correct programs.

What syntax analysis cannot do!

The information which syntax analysis phase gets from the previous phase (lexical analysis) is whether a token is valid or not and which class of tokens does it belong to. Hence it is beyond the capabilities of the syntax analysis phase to settle issues like:

- . Whether or not a variable has already been declared?
- . Whether or not a variable has been initialized before use?
- . Whether or not a variable is of the type on which the operation is allowed?

All such issues are handled in the **semantic analysis** phase

Limitations of regular languages

. How to describe language syntax precisely and conveniently. Can regular expressions be used?

. Many languages are not regular, for example, **string of balanced parentheses**

☞ - ((((.)))) and - { (i) i | i = 0 } : There is no regular expression for this language

Regular expressions cannot be used to describe language syntax precisely and conveniently. There are many languages which are not regular. For example, consider a language consisting of all strings of balanced parentheses. There is no regular expression for this language. Regular expressions can not be used for syntax analysis (specification of grammar) because: . The pumping lemma for regular languages prevents the representation of constructs like a string of balanced parenthesis where there is no limit on the number of parenthesis. Such constructs are allowed by most of the programming languages. . This is because a finite automaton may repeat states, however, it does not have the power to remember the number of times a state has been reached. . Many programming languages have an inherently recursive structure that can be defined by Context Free Grammars (CFG) rather intuitively. So a more powerful language is needed to describe valid string of tokens

A context free grammar has four components: <T, N, P, S>

- **A set of tokens**, known as **terminal symbols (T)** . Terminals are the basic symbols from which strings are formed (usually small letters).
- **A set of non-terminals (N)**: Non-terminals are syntactic variables that denote sets of strings. The non-terminals define sets of strings that help define the language generated by the grammar.(Capital letters)
- **A set of productions (P)**: The productions of a grammar specify the manner in which the terminals and non-terminals can be combined to form strings. Each production consists of a **non-terminal** called the **left side of the production,** an arrow, and a *sequence of tokens and/or on- terminals*, called the **right side of the production**.
- A designation of one of the non-terminals as **the start symbol (S)** , and the set of strings it denotes is the language defined by the grammar.

The strings are derived from the start symbol by repeatedly replacing a non-terminal (initially the start symbol) by the **right hand side** of a production for that **non-terminal.**

Example

Assume: S⟶ (S) S | ε is the grammar for a string of balanced parentheses.

For example, consider the string: (( ) ( )). It can be derived as:

Production Rules: P1: S⟶(S)S P2: S ⟶ ε

S⟶(**S**)S by P1

S ⟶ ((S)S)**S** by P1

S ⟶ ((**S**)S) by P2

S ⟶ (( )**S**) by P2

S ⟶ (( ) (**S**)S) by P1

S ⟶ (( )( )**S**) by P2

S ⟶ (( )( )) by P2 i.e. S ⟶ (( )( ))

Similarly,

Grammar: *list* ⟶ list + digit | list - digit | digit

digit ⟶ 0 | 1 | … | 9 is the grammar for a string of digits separated by + or -.

For example, consider the string 9 - 5 + 2 . It can be derived as:

list ⟶ list + digit

⟶ list - digit + digit

⟶ digit - digit + digit

⟶ 9 - digit + digit

⟶ 9 - 5 + digit

⟶ 9 - 5 + 2

It would be interesting to know that the name context free grammar comes from the fact that use of a production X ⟶ . does not depend on the context of X.

Syntax analyzers

. Testing for membership whether word(w) belongs to L(G) is just a "yes" or "no" answer

. However the syntax analyzer

- Must generate the parse tree and Handle errors gracefully if string is not in the language

. Form of the grammar is important

- Many grammars generate the same language

- Tools are sensitive to the grammar

A parse tree may be viewed as a graphical representation for a derivation that filters out the choice regarding replacement order. Each interior node of a parse tree is labeled by some non-terminal A , and that the children of the node are labeled, from left to right, by the symbols in the right side of the production by which this A was replaced in the derivation.

A syntax analyzer not only tests whether a construct is syntactically correct i.e. belongs to the language represented by the specified grammar but also generates the parse tree. It also reports appropriate error messages in case the string is not in the language represented by the grammar specified. It is possible that many grammars represent the same language. However, the tools such as *yacc (yet another compiler compiler)* or other parser generators are sensitive to the grammar form.

Derivation

If there is a production A ⟶ a then it is read as " A derives a " and is denoted by A $\Rightarrow$ a . The production tells us that we could replace one instance of an A in any string of grammar symbols by a .

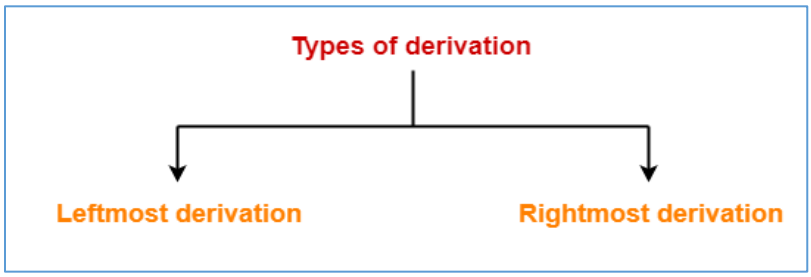
In a more abstract setting, we say that a A ß $\Rightarrow$ a γ ß if A ⟶ € is a production and a and ß are arbitrary strings of grammar symbols

If $a_1 \Rightarrow a_2 \Rightarrow . \Rightarrow a_n$ then we say a_1 derives a_n . The symbol $\Rightarrow$ means "derives in one step". Often we wish to say "derives in one or more steps". For this purpose, we can use the symbol $\Rightarrow$ with a + on its top.

Thus, if a string w of terminals belongs to a grammar G, it is written as S $\Rightarrow$ w . If S $\Rightarrow$ a , where a may contain non-terminals, then we say that a is a **sentential form of G**. A sentence is a sentential form with no non-terminals

What is Derivation?

- The process of deriving a string is called as **derivation**.
- The geometrical representation of a derivation is called as a **parse tree** or **derivation tree.**



1. Leftmost derivation

- The process of deriving a string by expanding the leftmost non-terminal at each step is called as **leftmost derivation**.
- The geometrical representation of leftmost derivation is called as a **leftmost derivation tree**

Example-

Consider the following grammar-

$$S \rightarrow aB \mid bA$$

$$A \rightarrow aS \mid bAA \mid a$$

$$B \rightarrow bS \mid aBB \mid b$$

Let us consider a string w = **aaabbabbba**

Now, let us derive the string w using leftmost derivation

Leftmost Derivation-

$S \rightarrow a\mathbf{B}$

$S \rightarrow aa\mathbf{B}B$ (Using $B \rightarrow aBB$)

$S \rightarrow aaa\mathbf{B}BB$ (Using $B \rightarrow aBB$)

$S \rightarrow aaab\mathbf{B}B$ (Using $B \rightarrow b$)

$S \rightarrow aaabb\mathbf{B}$ (Using $B \rightarrow b$)

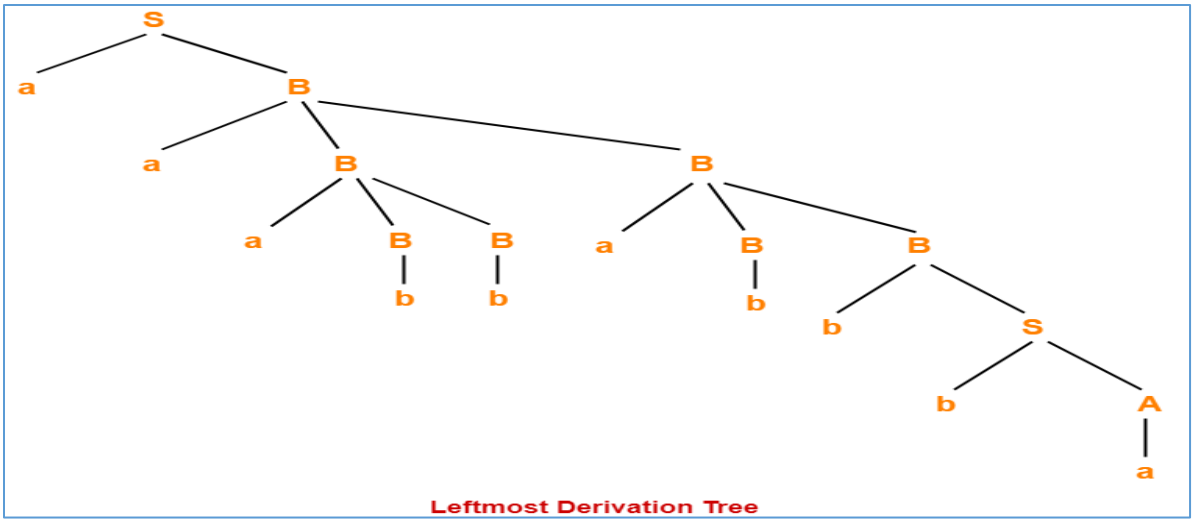
$S \rightarrow aaabba\mathbf{B}B$ (Using $B \rightarrow aBB$)

$S \rightarrow aaabbab\mathbf{B}$ (Using $B \rightarrow b$)

$S \rightarrow aaabbabb\mathbf{S}$ (Using $B \rightarrow bS$)

$S \rightarrow aaabbabbb\mathbf{A}$ (Using $S \rightarrow bA$)

$S \rightarrow aaabbabbba$ (Using $A \rightarrow a$)



Leftmost Derivation Tree

2. Rightmost Derivation-

The process of deriving a string by expanding the rightmost non-terminal at each step is called as **rightmost derivation**.

- The geometrical representation of rightmost derivation is called as a **rightmost derivation tree**.

Example- Consider the following grammar-

$$S \rightarrow aB \mid bA$$

$$A \rightarrow aS \mid bAA \mid a$$

$$B \rightarrow bS \mid aBB \mid b$$

Let us consider a string w = **aaabbabbba**

Now, let us derive the string w using rightmost derivation.

Rightmost Derivation-

S → a**B**

→ aaB**B** (Using B → aBB)

→ aaBaB**B** (Using B → aBB)

→ aaBaBb**S** (Using B → bS)

→ aaBaBbb**A** (Using S → bA)

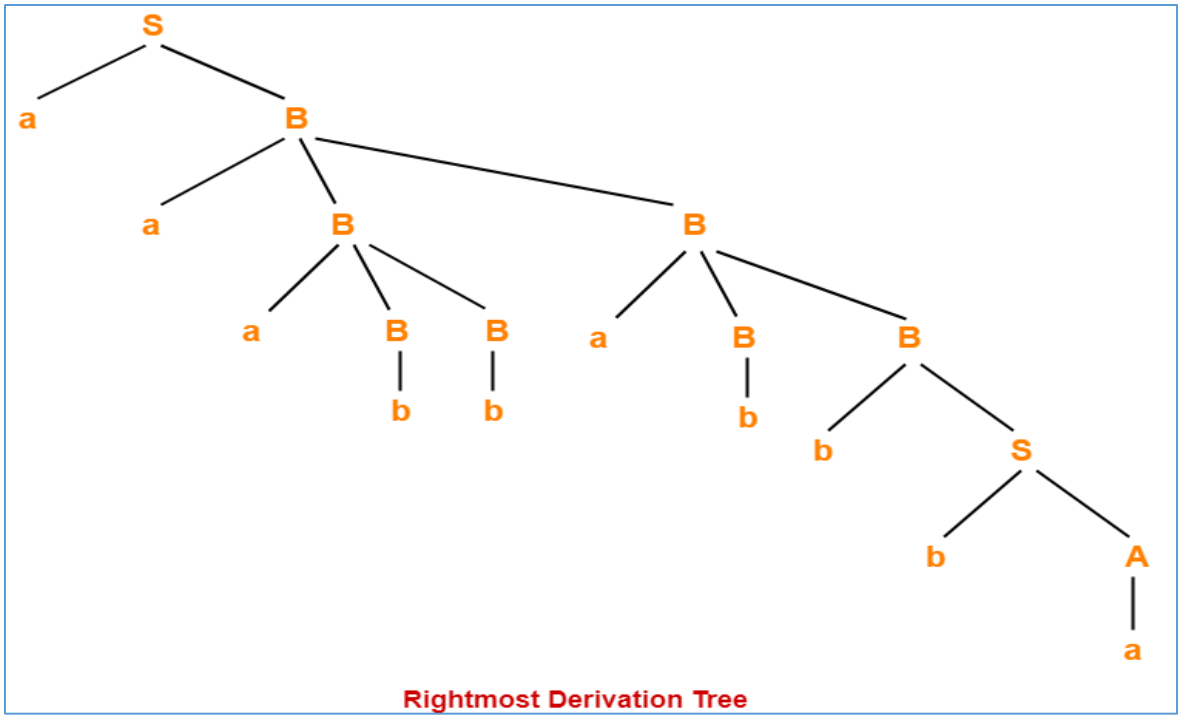
→ aaBa**B**bba (Using A → a)

→ aa**B**abbba (Using B → b)

→ aaaB**B**abbba (Using B → aBB)

→ aaa**B**babbba (Using B → b)

→ aaabbabbba (Using B → b)

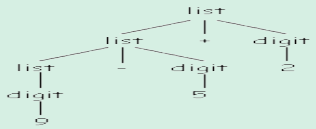


Parse tree

- It shows how the start symbol of a grammar derives a string in the language
- root is labeled by the start symbol
- leaf nodes are labeled by tokens
- Each internal node is labeled by a non terminal

A parse tree may be viewed as a graphical representation for a derivation that filters out the choice regarding replacement order. Thus, a parse tree pictorially shows how the start symbol of a grammar derives a string in the language. Each interior node of a parse tree is labeled by some **non-terminal A** , and that the children of the node are labeled, **from left to right**, by the symbols in the right side of the production by which this A was replaced in the derivation. The root of the parse tree is labeled by the start symbol and the leaves by non-terminals or terminals and, read from left to right, they constitute a sentential form, called the yield or frontier of the tree. So, if A is a non-terminal labeling an internal node and x1 , x2 , .xn are labels of children of that node then A $\longrightarrow$ x1 x2 . xn is a production.

Example **Parse tree for 9-5+2**



Production 1: list ⟶ list + digit

Production 2: list ⟶ list - digit

Production 3: list ⟶ digit

digit ⟶ 0|1|2|3|4|5|6|7|8|9

The parse tree for 9-5+2 implied by the derivation is shown.

. 9 is a *list* by production (3), since 9 is a digit.

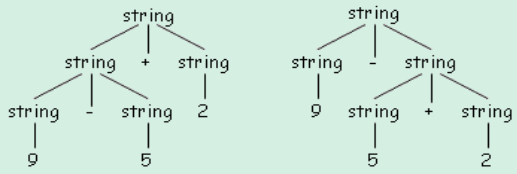
. 9-5 is a *list* by production (2), since 9 is a list and 5 is a digit.

. 9-5+2 is a *list* by production (1), since 9-5 is a list and 2 is a digit.

Ambiguity: A grammar is said to be an ambiguous grammar if there is some string that it can generate in more than one way (i.e., the string has more than one parse tree or more than one leftmost derivation). A language is inherently ambiguous if it can only be generated by ambiguous grammars.

For example, consider the following grammar: ***string ⟶ string + string | string – string | 0 | 1 | . | 9***

In this grammar, the string 9-5+2 has two possible parse trees as shown below.



Consider the parse trees for string 9-5+2, expression like this has more than one parse tree. The two trees for 9-5+2 correspond to the two ways of parenthesizing the expression: (9-5)+2 and 9-(5+2). The second parenthesization gives the expression the value 2 instead of 6.

Ambiguity is harmful to the intent of the program. The input might be deciphered in a way which was not really the intention of the programmer, as shown above in the 9-5+2 example. Though there is no general technique to handle ambiguity i.e., it is not possible to develop some feature which

automatically identifies and removes ambiguity from any grammar. However, it can be removed, broadly speaking, in the following possible ways:-

1) Rewriting the whole grammar unambiguously.

2) Implementing **precedence** and **associatively** rules in the grammar.

Associativity

If an operand has operator on both the sides, the side on which operator takes this operand is the associativity of that operator

+, -, *, / are left associative

^, = are right associative

A binary operation on a set S that does not satisfy the associative law is called non-associative. **A left-associative** operation is a non-associative operation that is conventionally evaluated from left to right i.e., operand is taken by the operator on the left side.

For example, **6*5*4 = (6*5)*4 and not 6*(5*4)**

6/5/4 = (6/5)/4 and not 6/(5/4)

A **right-associative** operation is a non-associative operation that is conventionally evaluated from right to left i.e., operand is taken by the operator on the right side.

For example, **6^5^4 => 6^(5^4) and not (6^5)^4)**

x=y=z=5 => x=(y=(z=5))

Following is the grammar to generate strings with left associative operators. (Note that this is left recursive and may go into infinite loop. But we will handle this problem later on by making it right recursive)

left ⟶ left + letter | letter and letter ⟶ a | b | .... | z

Precedence: Precedence is a simple ordering, based on either importance or sequence. One thing is said to "take precedence" over another if it is either regarded as more important or is to be performed first.

For example, consider the string a+5*2. It has two possible interpretations because of two different parse trees corresponding to (a+5)*2 and a+(5*2). But the * operator has precedence over the + operator. So, the second interpretation is correct. Hence, the precedence determines the correct interpretation.

Parsing

- *.Process of determination whether a string can be generated by a grammar*
- *Parsing falls in two categories:*
  - ***Top-down parsing**: Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals)*
  - ***Bottom-up parsing**: Construction of the parse tree starts from the leaf nodes (tokens or terminals of the grammar) and proceeds towards root (start symbol)*

Parsing is the process of analyzing a continuous stream of input (read from a file or a keyboard, for example) in order to determine its **grammatical structure** with respect to a given formal grammar. The task of the parser is essentially to determine if and how the input can be derived from the start symbol within the rules of the formal grammar. This can be done in essentially two ways:

❖ Top-down parsing ()

It can be viewed as an attempt to find a leftmost derivation of an input string. A parser can start with the start symbol (**The root**) and try to create the **nodes of the pars tree** in pre-order. Intuitively, the parser starts from the largest elements and breaks them down into incrementally smaller parts. It may require a **back-tracking (**It means, if one derivation of a production fails, syntax analyzer restarts process using different rules of same production. This technique may process input string more than once to determine right production.) **LL (*Left-right Scanning of Input , Left most Derivation***) and **Recursive descent** parsers are examples of top-down parsers.

➢ Recursive descent parser

Recursive Descent Parser uses the technique of Top-Down Parsing with **backtracking**. It can be defined as a Parser that uses the various recursive procedure to process the input string with backtracking. It can be simply performed using a Recursive language. The first symbol of the string of R.H.S of production will uniquely determine the correct alternative to choose

Eg Let us consider the following grammar:

S→cAd

A→ ab | a

Let the input string is w=cad ?

Look at the parse tree in below figure

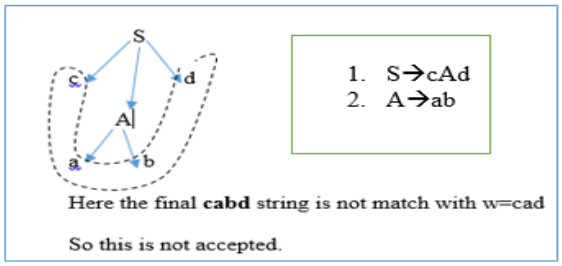


Fig. A

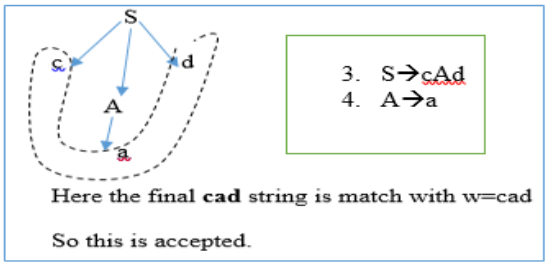


Fig. B

In **fig A** the parse tree is not correct since the final strings (nodes) are not match with the input string. Because of the production in A→ab is not suitable for the given input string. So we need go backward (back-tracking) on production A and rather than A→ab, we use the other alternative A→a (shown in **fig. B**) which is accepted.

- **Predictive Parser**

A Predictive Parser is a special case of Recursive Descent Parser, where no **Back-racking** is required. Predictive parsing relies on information about what first symbols can be generated by the right side of a production. The lookahead symbol guides the selection of the production A→ α to be used.

- if α starts with a token, then the production can be used when the lookahead symbol matches this token
- if α starts with a nonterminal *B*, then the production can be used if the lookahead symbol can be generated from *B*.

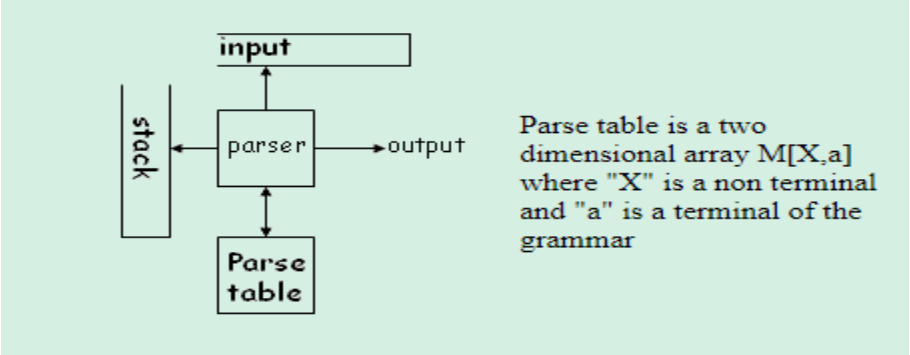
Algorithm 1

```
procedure match(a: token)
  if lookahead = a
    then lookahead = nexttoken
    else error
```

So match (a) moves the cursor lookahead one symbol forward **iff** lookahead points to a. Otherwise an error is produced.

Predictive parsing identifies what production to use to replace the input string. It does not have backtracking. The predicative parser uses a look ahead pointer. It points to the next input symbols. In order to make the parser free of backtracking.

❖ Predictive parser can be implemented by maintaining an external stack as shown below.



☞ **Reading Assignment: Stack implementation of Predictive Parsing and table construction**

❖ First and Follow

First and Follow sets are needed so that the parser can properly apply the needed production rule at the correct position.

✓ First

FIRST (α) is defined as the collection of terminal symbols which are the first letters of strings derived from α.

If X is Grammar Symbol, then First (X) will be −

- If X is a terminal symbol, then FIRST(X) = {X}
- If X → ε, then FIRST(X) = {ε}
- If X is non-terminal & X → a α, then FIRST (X) = {a}
- If X → Y_1, Y_2, Y_3, then FIRST (X) will be

(a) If Y is terminal, then

FIRST (X) = FIRST (Y_1, Y_2, Y_3) = {Y_1}

(b) If Y_1 is Non-terminal and

If Y_1 does not derive to an empty string i.e., If FIRST (Y_1) does not contain ε then, FIRST (X) = FIRST (Y_1, Y_2, Y_3) = FIRST(Y_1)

(c) If FIRST (Y_1) contains ε, then.

FIRST (X) = FIRST (Y_1, Y_2, Y_3) = FIRST(Y_1) − {ε} ∪ FIRST(Y_2, Y_3)

Similarly, FIRST (Y_2, Y_3) = {Y_2}, If Y_2 is terminal otherwise if Y_2 is Non-terminal then

- FIRST (Y_2, Y_3) = FIRST (Y_2), if FIRST (Y_2) does not contain ε.
- If FIRST (Y_2) contain ε, then
- FIRST (Y_2, Y_3) = FIRST (Y_2) − {ε} ∪ FIRST (Y_3)

Similarly, this method will be repeated for further Grammar symbols, i.e., for Y_4, Y_5, Y_6 … . Y_K.

Finally, if Y_k produce ε, ε will be added to First of Y_k,

✓ Follow

Follow (A) is defined as the collection of terminal symbols that occur directly to the right of A.

Rules to find FOLLOW

- If S is the start symbol, FOLLOW (S) ={$}
- If production is of form A → α B β, β ≠ ε.

(a) If FIRST (β) does not contain ε then, FOLLOW (B) = {FIRST (β)}

Or

(b) If FIRST (β) contains ε (i. e. , β ⇒* ε), then FOLLOW (B) = FIRST (β) − {ε} ∪ FOLLOW (A)

∵ when β derives ε, then terminal after A will follow B.

- If production is of form A → αB, then Follow (B) ={FOLLOW (A)}

Example 1 Consider the following grammar

S→ABCDE

A→a | ε

B→b | ε

C→c

D→d | ε

E→e | ε

First

First(S) ={ a, b, c }

First(A) = { a, ε }

First(B) = { b, ε }

First(C) = {c}

First(D) = {d, ε }

First (E) = { e, ε }

Follow

Follow (S) = {$}

Follow (A) = {b, c} i.e Follow A=First of B except ε

Follow(B) ={c}

Follow(C) = {d, e, $}

Follow (D) = {e, $}

Follow (E) = {$}

Example 2 consider the following grammar

S→ACB | CbB | Ba

A→da | BC

B→g | ε

C→h | ε

First

First (S) = { d, g, h, ε ,b, a}

First (A) = {d, g, h, ε }

First (B) = {g, ε}

First (C) = {h, ε}

Follow

Follow(S) = {$}

Follow (A) = {h, g, $}

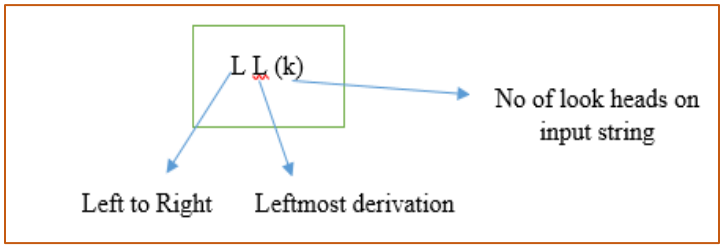
Follow (B) = {$, a, h, g}

Follow(C) = {g, $, b, h}

Important Notes-

- ∈ may appear in the first function of a non-terminal.
- ∈ will never appear in the follow function of a non-terminal.
- Before calculating the first and follow functions, eliminate **Left Recursion** from the grammar, if present.
- We calculate the follow function of a non-terminal by looking where it is present on the RHS of a production rule.

❖ LL(k) parsing techniques



If k=1 it is LL(1) grammar that we only look ahead one input string at a time. It is used in top down parsing. Consider the following grammar

1. S→ aA | Bb
2. A→ aB | Cb
3. b→bC | aC
4. C→bD
5. D→d

Consider the input string w=**aaabd**

The LL(1) derivation will look by taking 1 input string at atime

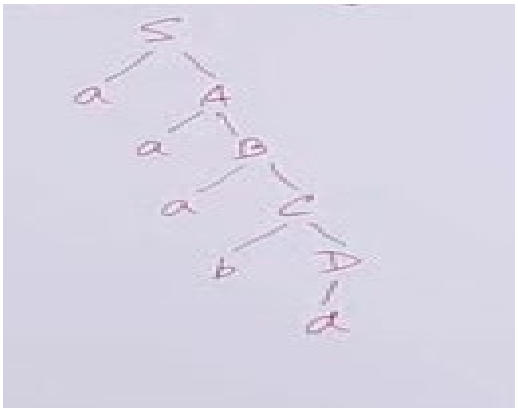
S→aA for 1st letter w=**a**aabd

A→aaB using rule 2 for 2nd letter w=**aa**abd

B→aaaC using rule 3 for 3rd letter w=**aaa**bd

C→aaabD using rule 4 for 4th letter w=**aaab**d

D→aaabd using rule 5 for last letter w=**aaabd**



Hence w=**aaabd** is LL(1) grammar based on the productions.

If it is not deterministic to select a single input string, then it is not **LL(1)** parsing

Example :

S→ abB | aaA

B→d

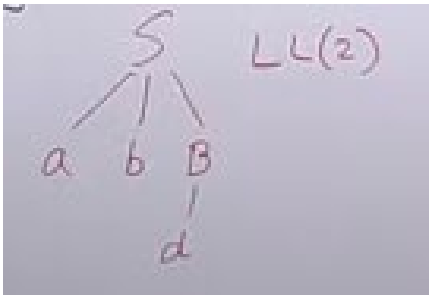
A→c | d

Let input string w=**abd**

At the start symbol S we have two possibilities that the first (a) input string can be derived (either S→abB or S→aaA) . This leads ambiguity. If we look by two letters at a time we can drive the **w=abd** as follows:

S→abB for **ab**d

B→abd for abd



Hence w=*abd* is not LL(1) grammer but is LL(2) grammer.

Note: **LL(k) C LL(K+1)** (C is subset)

LL (1) ⇨ **LL(2)** ⇨ **LL(3)** ⇨ **…** ⇨ **LL(k)** and the revers is not true

❖ Recursion

Grammar may be Left recursion or Right recursion.

☞ Left recursion

- A production of grammar is said to have **left recursion** if the leftmost variable of its RHS is same as variable of its LHS.
- A grammar containing a production having left recursion is called as Left Recursive Grammar.

Example-

$$S \rightarrow Sa / \in$$

(**Left Recursive Grammar**)

- Left recursion is considered to be a problematic situation for Top down parsers.
- Therefore, left recursion has to be eliminated from the grammar

Elimination of Left Recursion

Left recursion is eliminated by converting the grammar into a right recursive grammar.

If we have the left-recursive pair of productions-

$$\mathbf{A \rightarrow A\alpha / \beta}$$

where β does not begin with an A.

Then, we can eliminate left recursion by replacing the pair of productions with-

$$\mathbf{A \rightarrow \beta A'}$$

$$\mathbf{A' \rightarrow \alpha A' / \in}$$

(Right Recursive Grammar)

This right recursive grammar functions same as left recursive grammar.

Consider the following grammar and eliminate left recursion-

$$A \rightarrow ABd / Aa / a$$

$$B \rightarrow Be / b$$

The grammar after eliminating left recursion is-

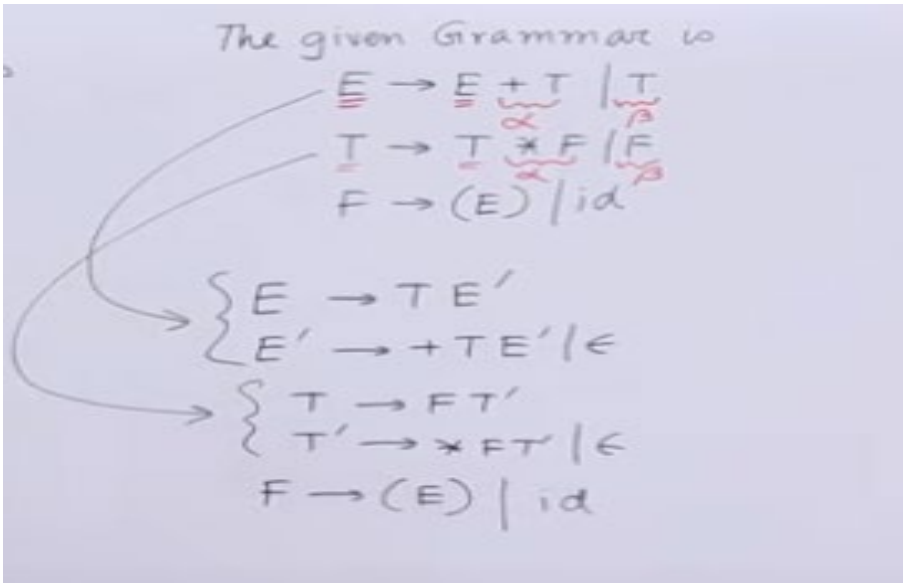
$A \rightarrow aA'$

$A' \rightarrow BdA' / aA' / \in$

$B \rightarrow bB'$

$B' \rightarrow eB' / \in$

Another example



☞ **Right Recursion**

- A production of grammar is said to have **right recursion** if the rightmost variable of its RHS is same as variable of its LHS.
- A grammar containing a production having right recursion is called as Right Recursive Grammar.

Example-

$$S \rightarrow aS / \in \qquad \text{(Right Recursive Grammar)}$$

- Right recursion does not create any problem for the Top down parsers.
- Therefore, there is no need of eliminating right recursion from the grammar

❖ Left factoring

Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing. The basic idea is that when it is not clear which of two or more alternative productions to use to expand a non-terminal A, we defer the decision till we have seen enough input to make the right choice.

In general if $A \longrightarrow \alpha ß_1 \mid \alpha ß_2$, we defer decision by expanding A to a A'.

and then we can expand A ' to $ß_1$ or $ß_1$

Therefore $A \longrightarrow \alpha ß_1 \mid \alpha ß_2$ transforms to

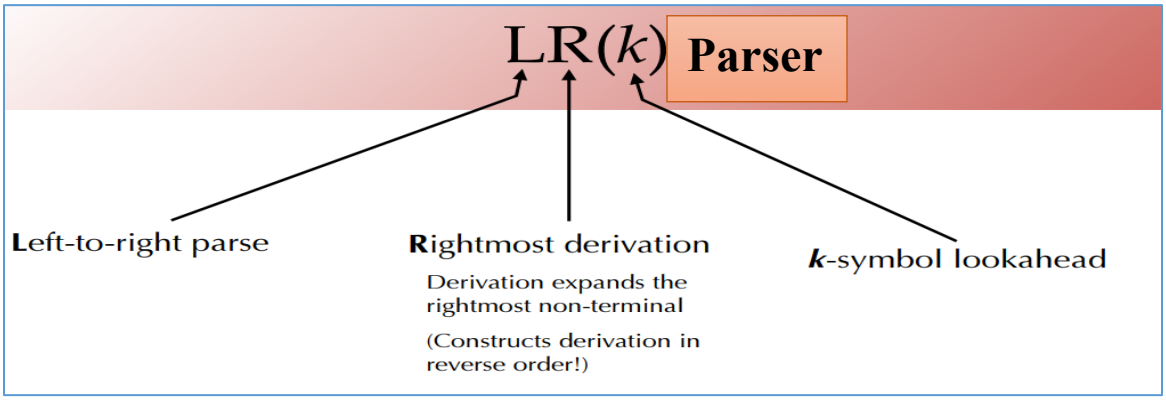
$A \longrightarrow \alpha A'$

$A' \longrightarrow ß_1 \mid ß_2$

❖ Bottom-up parsing

A parser can start with the input and attempt to rewrite it to the start symbol. **LR (**Left to Right Scanning of Input, **R**ightmost Derivation) parsers are examples of bottom-up parsers.

We will now study a general style of bottom up parsing, also known as shift-reduce parsing. Shift-reduce parsing attempts to construct a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top). We can think of the process as one of "reducing" a string w to the start symbol of a grammar. At each reduction step, a particular substring matching the right side of a production is replaced by the symbol on the left of that production, and if the substring is chosen correctly at each step, a rightmost derivation is traced out in reverse.



Basic idea: LR parser has a stack and input

Given contents of stack and k tokens look-ahead parser does one of following operations.

Stack	Input	**Actions**
$	w$	1. **"Shit":** Parser shifts zero or more symbols to the stack until handle β
.	.	2. **"Reduce":** β is reduced to left hand side of the production
.	.	3. **"Accept":** announce successful production
.	.	4. **"Error":** Calls an error recovery routine
$S	$	

Eg. Let Production rule *A →abc* here **abc** is considered as handle β and A is Left side of the production

- The set of strings to be **replaced** at each reduction step is called a handle.
- Although a handle of a string can be described as a substring that equals the right side of a production rule

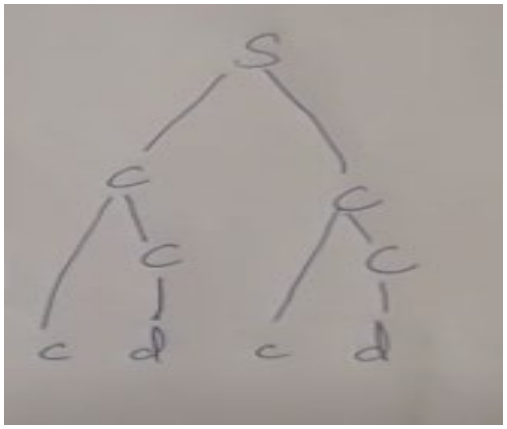
For example, let us take

S→CC

C→cC | d

Let us pares the input string w=**cdcd**

Stack	Input	Action
$	cdcd$	Shift
$c	dcd$	Shift since c is not handle
$cd	cd$	Reduce by C→d since d is handle
$cC	cd$	Reduce by C→cC since cC is handle
$C	cd$	Shift since C is not handle
$Cc	d$	Shift since C or c or Cc are not handle
$Ccd	$	Reduce by C→d since d is a handle
$CcC	$	Reduce by C→ cC since cC is handle
$CC	$	Reduce by S→CC since CC is handle
$S	$	Accepted



Parse tree

Example 2, consider the grammar

S → aABe
A → Abc | b
B → d

The sentence **abbcde** can be reduced to S by the following steps:

a b b c d e
a A b c d e
a A d e
a A B e
S

These reductions trace out the following right-most derivation in reverse:

S→ a A B e
S→a A d e
S→a A b c d e
S→a b b c d e

☞ **LR Parser** can be LR **SLR**(Simple LR) **LALR**(Look Ahead LR) **CLR**(Canonical LR)

How does the LR(k) parser know when to shift and to reduce?

- Uses a DFA
- At each step, parser runs DFA using symbols on stack as input
- Input is sequence of terminals and non-terminals from bottom to top
- Current state of DFA plus next k tokens indicate whether to **shift** or **reduce**

Error Recovery

A parser should be able to detect and report any error in the program. It is expected that when an error is encountered, the parser should be able to handle it and carry on parsing the rest of the input. Mostly it is expected from the parser to check for errors but errors may be encountered at various stages of the compilation process. A program may have the following kinds of errors at various stages:

- **Lexical** : name of some identifier typed incorrectly
- **Syntactical** : missing semicolon or unbalanced parenthesis
- **Semantical** : incompatible value assignment
- **Logical** : code not reachable, infinite loop

There are four common error-recovery strategies that can be implemented in the parser to deal with errors in the code.

Panic mode

When a parser encounters an error anywhere in the statement, it ignores the rest of the statement by not processing input from erroneous input to delimiter, such as semi-colon. This is the easiest way of error-recovery and also, it prevents the parser from developing infinite loops.

Statement mode

When a parser encounters an error, it tries to take corrective measures so that the rest of inputs of statement allow the parser to parse ahead. For example, inserting a missing semicolon, replacing comma with a semicolon etc. Parser designers have to be careful here because one wrong correction may lead to an infinite loop.

Error productions

Some common errors are known to the compiler designers that may occur in the code. In addition, the designers can create augmented grammar to be used, as productions that generate erroneous constructs when these errors are encountered.

Global correction

The parser considers the program in hand as a whole and tries to figure out what the program is intended to do and tries to find out a closest match for it, which is error-free. When an erroneous input (statement) X is fed, it creates a parse tree for some closest error-free statement Y. This may allow the parser to make minimal changes in the source code, but due to the complexity (time and space) of this strategy, it has not been implemented in practice yet.